

Promises & async/await

*Usage pratique des Promises,
avec approfondissement théorique sur async/await*

Préparé pour un profil Symfony / JS-jQuery / React (Preact)

1. Le problème que les Promises résolvent

1.1 Avant les Promises : les callbacks

```
// Style callback (ancien, encore visible dans certaines APIs Node.js ou jQuery)
recupererUtilisateur(id, function(erreur, utilisateur) {
  if (erreur) {
    console.error(erreur);
    return;
  }
  recupererCommandes(utilisateur.id, function(erreur, commandes) {
    if (erreur) {
      console.error(erreur);
      return;
    }
    recupererDetailCommande(commandes[0].id, function(erreur, detail) {
      // et ainsi de suite... "callback hell" ou "pyramide de la mort"
    });
  });
});
```

Chaque opération asynchrone imbrique la suivante dans son callback de succès — illisible au-delà de 2-3 niveaux, et la gestion d'erreur doit être dupliquée à chaque étage.

1.2 Avec les Promises : un objet qui représente "une valeur pas encore là"

Une **Promise** est un objet qui représente le résultat (futur) d'une opération asynchrone. Elle peut être dans 3 états :

- **pending** (en attente) — l'opération n'est pas encore terminée
- **fulfilled** (résolue) — l'opération a réussi, une valeur est disponible
- **rejected** (rejetée) — l'opération a échoué, une erreur est disponible

Une fois **fulfilled** ou **rejected**, une Promise ne change plus jamais d'état (elle est dite "settled" / réglée).

```
recupererUtilisateur(id)
  .then(utilisateur => recupererCommandes(utilisateur.id))
  .then(commandes => recupererDetailCommande(commandes[0].id))
  .then(detail => console.log(detail))
  .catch(erreur => console.error(erreur)); // UNE SEULE gestion d'erreur pour toute la chaîne
```

2. Créer et utiliser une Promise

2.1 Créer une Promise "à la main"

```
function attendre(milliseconides) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve('Terminé après ' + milliseconides + 'ms');
    }, milliseconides);
  });
}

attendre(1000).then(message => console.log(message));
```

Le constructeur `new Promise((resolve, reject) => { ... })` prend une fonction (l'"executor") qui reçoit deux fonctions : - `resolve(valeur)` — appelée quand l'opération réussit, avec le résultat - `reject(erreur)` — appelée quand l'opération échoue, avec l'erreur

En pratique, tu écris rarement `new Promise(...)` toi-même au quotidien — la plupart des APIs modernes (`fetch`, `fs.promises` en Node, etc.) retournent déjà des Promises prêtes à l'emploi. Le savoir construire reste utile pour encapsuler une API à callback ancienne, ou pour des cas comme un `attendre()` / `sleep()`.

2.2 `.then()`, `.catch()`, `.finally()`

```
fetch('/api/articles')
  .then(reponse => reponse.json()) // transforme la réponse brute en JSON
  .then(donnees => console.log(donnees)) // utilise les données
  .catch(erreur => console.error('Erreur :', erreur)) // capture TOUTE erreur de la chaîne au-dessus
  .finally(() => console.log('Requête terminée')); // s'exécute toujours, succès ou échec
```

Point essentiel : `.catch()` capture une erreur survenue **n'importe où** dans la chaîne `.then()` précédente — pas seulement la dernière étape. C'est l'équivalent asynchrone d'un `try/catch` qui englobe tout un bloc.

2.3 Piège classique n°1 : oublier de retourner dans un `.then()`

```
// FAUX – la Promise suivante reçoit `undefined`, pas les commandes
fetch('/api/utilisateur')
  .then(reponse => reponse.json())
  .then(utilisateur => {
    fetch(`/api/commmandes/${utilisateur.id}`); // pas de `return` !
  })
  .then(commmandes => {
    console.log(commmandes); // undefined, car rien n'a été retourné à l'étape précédente
  });

// CORRECT
fetch('/api/utilisateur')
  .then(reponse => reponse.json())
  .then(utilisateur => {
    return fetch(`/api/commmandes/${utilisateur.id}`); // le `return` chaîne bien la Promise suivante
  })
  .then(reponseCommmandes => reponseCommmandes.json())
  .then(commmandes => {
    console.log(commmandes); // les vraies données, cette fois
  });
```

Si tu retournes une Promise dans un `.then()`, la chaîne **attend** automatiquement qu'elle se résolve avant de passer à l'étape suivante — c'est ce qui permet l'enchaînement propre. Sans `return`, le `.then()` suivant reçoit `undefined` immédiatement, sans attendre.

2.4 Piège classique n°2 : créer des Promises imbriquées au lieu de chaîner

```
// FAUX - réintroduit le "callback hell" qu'on cherchait justement à éviter
recupererUtilisateur(id).then(utilisateur => {
  recupererCommandes(utilisateur.id).then(commandes => {
    recupererDetail(commandes[0].id).then(detail => {
      console.log(detail);
    });
  });
});

// CORRECT - chaînage à plat
recupererUtilisateur(id)
  .then(utilisateur => recupererCommandes(utilisateur.id))
  .then(commandes => recupererDetail(commandes[0].id))
  .then(detail => console.log(detail));
```

3. Exécuter plusieurs Promises en parallèle

3.1 `Promise.all` — attendre que TOUTES réussissent

```
const [utilisateur, commandes, notifications] = await Promise.all([
  fetch('/api/utilisateur').then(r => r.json()),
  fetch('/api/commandes').then(r => r.json()),
  fetch('/api/notifications').then(r => r.json()),
]);
```

Les trois requêtes partent **en même temps** (pas l'une après l'autre) — un gain de temps important par rapport à un enchaînement séquentiel quand les opérations sont indépendantes. `Promise.all` se résout avec un tableau des résultats, **dans le même ordre** que les Promises d'entrée (peu importe l'ordre réel de complétion).

Piège : si **une seule** Promise du tableau est rejetée, `Promise.all` rejette **immédiatement** l'ensemble, même si les autres Promises sont en cours ou réussiraient — tu perds l'accès aux résultats des autres.

3.2 `Promise.allSettled` — attendre TOUTES, qu'elles réussissent ou échouent

```
const resultats = await Promise.allSettled([
  fetch('/api/service-a'),
  fetch('/api/service-b'), // même si celui-ci échoue...
  fetch('/api/service-c'), // ...celui-ci sera quand même traité
]);

resultats.forEach((resultat, index) => {
  if (resultat.status === 'fulfilled') {
    console.log(`Service ${index} OK :`, resultat.value);
  } else {
    console.log(`Service ${index} échoué :`, resultat.reason);
  }
});
```

Utile quand tu veux le résultat de chaque opération indépendamment, sans qu'un échec n'empêche de connaître le résultat des autres — typiquement pour agréger plusieurs sources de données dont certaines peuvent être temporairement indisponibles.

3.3 `Promise.race` — la première qui se termine (succès OU échec)

```
const resultat = await Promise.race([
  fetch('/api/donnees'),
  attendre(5000).then(() => { throw new Error('Timeout'); }),
]);
```

Pattern courant : implémenter un timeout sur une requête réseau, en faisant "courir" la vraie requête contre un délai qui échoue après X millisecondes.

4. `async` / `await` — la syntaxe moderne

4.1 Le principe : `async` / `await` est du sucre syntaxique au-dessus des Promises

```
// Avec .then() classique
function recupererDonnees() {
  return fetch('/api/articles')
    .then(reponse => reponse.json())
    .then(donnees => donnees.filter(d => d.actif));
}

// Avec async/await — STRICTEMENT ÉQUIVALENT, juste plus lisible
async function recupererDonnees() {
  const reponse = await fetch('/api/articles');
  const donnees = await reponse.json();
  return donnees.filter(d => d.actif);
}
```

Ce qu'il faut bien comprendre : une fonction déclarée `async` retourne **toujours** une Promise, même si tu écris un simple `return valeur;` à l'intérieur — JavaScript l'enveloppe automatiquement dans une Promise résolue.

```
async function exemple() {
  return 42;
}

exemple(); // ne retourne PAS 42, retourne une Promise<42>
exemple().then(v => console.log(v)); // affiche 42
```

`await` ne fonctionne **qu'à l'intérieur** d'une fonction marquée `async` (sauf au niveau racine d'un module ES, "top-level await", cas plus rare). `await maPromise` suspend l'exécution de la fonction jusqu'à ce que `maPromise` soit résolue, puis "déballe" sa valeur — ou relance l'erreur si elle est rejetée.

4.2 Gestion des erreurs : `try` / `catch`, pas `.catch()`

```
async function recupererDonnees() {
  try {
    const reponse = await fetch('/api/articles');
    if (!reponse.ok) {
      throw new Error(`Erreur HTTP : ${reponse.status}`);
    }
    return await reponse.json();
  } catch (erreur) {
    console.error('Échec de la récupération :', erreur);
    throw erreur; // souvent utile de relancer, pour que l'appelant sache aussi que ça a échoué
  }
}
```

Un `await` sur une Promise rejetée se comporte exactement comme un `throw` classique — donc la gestion d'erreur redevient du `try` / `catch` JS standard, plus familier qu'une chaîne de `.catch()`.

Piège fréquent : oublier le `try` / `catch` autour d'un `await` fait planter silencieusement la fonction (la Promise retournée par la fonction `async` est rejetée, mais si rien ne l'attrape plus haut, ça devient une "unhandled promise rejection" — visible en console, mais souvent ignorée en développement).

4.3 Ce qui se passe réellement : la mécanique d'exécution (la partie théorique demandée)

C'est le point qui clarifie le plus durablement ce qu'on "tente sans comprendre" avec `async/await`. Voici le déroulé précis.

JavaScript est mono-thread : une seule chose s'exécute à la fois. Quand le moteur JS rencontre `await`, il ne "bloque" pas le thread (contrairement à ce que la syntaxe pourrait suggérer visuellement) — il **suspend la fonction async courante**, et **rend immédiatement la main** à ce qui a appelé cette fonction, pour continuer à exécuter le reste du code synchrone qui suit.

```
console.log('1');

async function exemple() {
  console.log('2');
  await attendre(1000);
  console.log('4'); // ne s'exécute qu'après l'attente, ET après tout le code synchrone restant
}

exemple();
console.log('3');

// Ordre d'affichage réel : 1, 2, 3, 4
```

Pourquoi 3 s'affiche avant 4, alors que `exemple()` est appelée avant le `console.log('3')` : dès que `exemple()` atteint le `await`, elle se suspend et rend le contrôle au code appelant — qui continue avec `console.log('3')` immédiatement (sans attendre que les 1000ms se soient écoulées). Ce n'est qu'**après** que tout le code synchrone restant du programme s'est exécuté que JavaScript reprend l'exécution de `exemple()` là où elle s'était arrêtée.

La micro-tâche (microtask) et la boucle d'événements (event loop) : quand une Promise se résout, sa continuation (la suite du code après le `await`, ou le callback passé à `.then()`) n'est pas exécutée immédiatement — elle est placée dans une **file de microtâches**. JavaScript termine d'abord d'exécuter tout le code synchrone en cours (jusqu'à la fin de la "pile d'appels" actuelle), puis vide entièrement la file de microtâches, avant de passer à autre chose (comme un nouveau timer, un événement utilisateur, etc.).

```
console.log('A');

setTimeout(() => console.log('B'), 0); // macrotâche (timer)

Promise.resolve().then(() => console.log('C')); // microtâche

console.log('D');

// Ordre d'affichage réel : A, D, C, B
// Pas A, B, C, D comme l'ordre d'écriture pourrait le suggérer !
```

Explication : **A** et **D** sont synchrones, donc exécutés immédiatement dans l'ordre. `setTimeout` (même à 0ms) et le `.then()` de la Promise sont tous deux asynchrones — mais les **microtâches** (Promises) sont **toujours vidées avant** les **macrotâches** (timers, événements DOM, I/O) à chaque "tour" de la boucle d'événements. D'où **C** avant **B**.

Pourquoi cette mécanique compte concrètement : c'est ce qui explique des comportements qu'on "subit" parfois sans comprendre — par exemple un état React mis à jour dans un `.then()` qui semble "arriver après" un autre code qu'on pensait pourtant exécuté plus tard dans le fichier. Une fois cette règle internalisée (synchrone d'abord, puis microtâches/Promises, puis macrotâches/timers), ce genre de surprise disparaît.

4.4 `await` en boucle : le piège de performance le plus fréquent

```
// LENT – chaque itération ATTEND la précédente avant de démarrer la suivante,
// alors que les requêtes sont indépendantes entre elles
async function recupererTout(ids) {
  const resultats = [];
  for (const id of ids) {
    const reponse = await fetch(`/api/articles/${id}`); // séquentiel !
    resultats.push(await reponse.json());
  }
}
```

```

    }
    return resultats;
}

// RAPIDE – toutes les requêtes partent en même temps
async function recupererTout(ids) {
    const promesses = ids.map(id =>
        fetch(`/api/articles/${id}`).then(r => r.json())
    );
    return Promise.all(promesses); // attend que toutes se terminent, en parallèle
}

```

Si les opérations sont **indépendantes** (le résultat de l'une n'est pas nécessaire pour démarrer l'autre), un `await` dans une boucle `for` les rend artificiellement séquentielles — c'est souvent un vrai problème de performance en production, pas juste un détail de style.

Exception légitime : si chaque itération **dépend** du résultat de la précédente (pagination où l'URL de la page suivante vient de la réponse précédente, par exemple), alors le séquentiel via `await` en boucle est non seulement correct, mais nécessaire.

4.5 `async / await` avec `Promise.all` — le pattern à retenir

```

async function chargerTableauDeBord() {
    try {
        const [utilisateur, statistiques, notifications] = await Promise.all([
            fetch('/api/utilisateur').then(r => r.json()),
            fetch('/api/statistiques').then(r => r.json()),
            fetch('/api/notifications').then(r => r.json()),
        ]);

        return { utilisateur, statistiques, notifications };
    } catch (erreur) {
        // Attention : si UNE seule requête échoue, on atterrit ici,
        // sans savoir laquelle ni avoir les résultats des autres
        console.error('Erreur de chargement : ', erreur);
        throw erreur;
    }
}

```

C'est la combinaison la plus fréquente en pratique : `async / await` pour la lisibilité générale, `Promise.all` pour le parallélisme, `try / catch` pour la gestion d'erreur centralisée.

5. Exemple complet : applique tout ce qui précède

Un scénario réaliste, proche de ta synchronisation EBP/API : récupérer une liste d'interventions, puis pour chacune, récupérer son détail, avec gestion d'erreur et parallélisation contrôlée.

```
async function synchroniserInterventions(salonId, jour) {
  // Étape 1 : récupérer la liste (séquentiel, car on a besoin du résultat avant de continuer)
  let interventions;
  try {
    const reponseListe = await fetch(`/api/interventions?salon=${salonId}&jour=${jour}`);
    if (!reponseListe.ok) {
      throw new Error(`Erreur liste interventions : ${reponseListe.status}`);
    }
    interventions = await reponseListe.json();
  } catch (erreur) {
    console.error('Impossible de récupérer la liste : ', erreur);
    return { succes: false, erreur };
  }

  // Étape 2 : récupérer le détail de chaque intervention EN PARALLÈLE
  // (les détails sont indépendants entre eux, donc pas besoin de séquentiel)
  const resultatsDetails = await Promise.allSettled(
    interventions.map(intervention =>
      fetch(`/api/interventions/${intervention.id}/detail`).then(r => r.json())
    )
  );

  // Étape 3 : séparer les succès des échecs, plutôt que de tout faire échouer
  // au premier problème (d'où le choix de allSettled plutôt que all)
  const details = [];
  const erreurs = [];

  resultatsDetails.forEach((resultat, index) => {
    if (resultat.status === 'fulfilled') {
      details.push(resultat.value);
    } else {
      erreurs.push({ interventionId: interventions[index].id, raison: resultat.reason });
    }
  });

  return { succes: true, details, erreurs };
}
```

Ce code illustre les choix à faire consciemment : séquentiel quand une étape dépend de la précédente (étape 1 → 2), parallèle quand les opérations sont indépendantes (tous les détails de l'étape 2 entre eux), et `allSettled` plutôt que `all` quand on préfère récupérer un maximum de résultats partiels plutôt que de tout perdre au premier échec.

Synthèse : les règles à toujours appliquer

Situation	Bonne pratique
Plusieurs opérations indépendantes	<code>Promise.all</code> (ou <code>allSettled</code> si les échecs partiels sont acceptables)
Une opération dépend du résultat de la précédente	<code>await</code> séquentiel (boucle <code>for</code> ou chaîne explicite)
Tout code utilisant <code>await</code>	L'envelopper dans <code>try / catch</code>
<code>await</code> dans une boucle sur des opérations indépendantes	Remplacer par <code>.map()</code> + <code>Promise.all</code>
Un <code>.then()</code> qui appelle une autre opération <code>async</code>	Toujours <code>return</code> la Promise interne
Comprendre un ordre d'exécution qui semble illogique	Se rappeler : synchrone d'abord, puis microtâches (Promises), puis macrotâches (timers)